C-MCPN: The Clingo McCulloch-Pitts Network Editor

An Answer-Set Programming Framework for Artificial Neural Networks

David Gonzalez[1], Joshua T. Guerin, Ph.D.[2]
Mathematical Sciences
University of Northern Colorado
Greeley, CO 80639
[1]********@bears.unco.edu
[2]********@unco.edu

Abstract

As Artificial Intelligence and generative AI continue to engage the public in ways that mirror other critical advances (e.g., the PC and smartphone revolutions as well as the broader Internet) public interest has turned more and more towards applications of the underlying technologies and architectures that power them. While machine learning technologies like LLMs have a profound capacity to integrate AI into modern life, the underlying artificial neural network technologies present significant concern for both short and long-term sustainability: data center footprint, waste heat and noise, and infrastructure/resources for energy and clean water.

At the same time, advanced declarative languages have made remarkable headway towards AI solutions are considerably more efficient. In particular, Answer Set Programming languages are designed to solve computationally difficult (e.g., NP-Complete and NP-Hard) problems with limited available resources. In this paper we outline how these languages can be used to model artificial neural network architectures, like those used in LLMs, in the modeling language Clingo. We also introduce the C-MCPN (Clingo McCulloch-Pitts Network) Framework and Editor– a tool largely written in Python and Clingo. C-MCPN was developed to support our work in generating more efficient neural networks by providing a graphical interface to work with these models, using Clingo as a back-end for both representation and inference.

1 Introduction

Artificial Intelligence systems have had a significant impact on the modern world over the decades. Some of the many advancements across various fields include conversational agents, marketing, medicine, and insurance **Nössig et al., 2024–need cite**. Unlike the impacts of AI in previous decades, general public awareness has skyrocked due to advances in generative AI (GEN-AI), in particular techniques from machine learning (ML) like the artificial neural networks (ANNs) that power contemporary large language models (LLMs).

While these AI systems have the potential for significant advancement, the underlying technologies come with significant drawbacks: footprints of data centers (which has garnered particular public interest–**cite here?**), worsening use of finite resources such as fresh water and electricity, and the potential for significant environmental and health impacts **Bush 2025?**.

Conversely, Answer Set Programming (ASP) languages often live at a different end of the research spectrum in terms of their efficiency. These languages are designed to solve NP-Complete and NP-Hard problems in a way that is fast and efficient on limited hardware such as consumer computers. Our current work involves exploring the use of ASP languages to accelerate the processes of learning and inference, with a hope of cutting into the resources needed for these technologies to continue growing in use.

In this paper we demonstrate that ANNs, specifically McCullough-Pitts Networks (MPNs) can be used to model and conduct inference over ANNs. We also introduce `C-MCPN`: The *Clingo McCullouch-Pitts Network Editor*. C-MCPN is a tool that we have developed to assist in the exploration of ASP-based networks, and allows the user to model network architectures, including the underlying representations and inference over ANNs.

2 Background

Our work is related to the merging of ANN technology with more efficient Answer Set Programming languges. In this section we will provide a brief background on both subjects, motivating our work on C-MCPN.

2.1 Artificial Neural Networks (ANNs)

Note: *As far as I can tell we are basically missing this section in the other paper(?), which we will need. In the other paper, section 2.3, there are a lot of citations and descriptions around the general subject, but nothing like "here is the definition of an ANN as an example."*

2.1.1 Efficiency and Environmental Immpacts

The introduction of big data (in particular by scraping the broader Internet) changed the landscape for AI, allowing statistical models to flex their strength, in particular because these models require massive amounts of data for training purposes [**Helm2020**, **Bhuyan2024**]. This incredible need for information highlights an important drawback in the form of *data efficiency* [**Cropper2020**]. This notable lack of efficiency translates to a significant need in terms of resources.

As an example, let us consider a the cost to train a single model, ChatGPT-3. According to DeVries **(et al.? I can't find the bib right now.)**, the training of GPT-3 required around 1.3 million kWh *just* to be trained [**DeVries2023**]. With some quick, back-of-the-envelope calculations, we can put this into perspective. Consider that the average American household uses approximately 10,791 kWh per year **is the source EIA2025?**. Dividing these out we find that the model's energy use was proportional to:

$$1{,}300{,}000 \text{ kWh} \times \frac{1 \text{ household}}{10{,}791 \text{ kWh/year}} \approx 120.5 \text{ households.}$$

Similarly, we could produce a rough, *lower bound* of how much water was likely used for training. A termoelectric power station can use approximately 43.8 L/kWh of water [**Gordon2024**]. That would put the *indirect* water costs of training GPT-3 at approximately 56,940,000 L, approximated by:

$$1{,}300{,}000 \text{ kWh} \times 43.8 \frac{\text{L}}{\text{kWh}} = 56{,}940{,}000 \text{ L}$$

Given that an adult male averages approximately 4L of fresh water, that number represents sufficient water to sustain a lower bound of approximately 39,000 adults for a year **Citations needed: 4L., possible arithmetic needed to spell out 39K**. To put this number into perspective, this would likely be sufficient to provide fresh water to approximately the city of Brighton, CO, for a year.

Note that this figure includes only the indirect costs to generate the electricity– massive cooling costs for data center equipment would likely add significantly to our calculation. While these calculations are remarkably overly-simplified, we hope that it puts into perspective how the technology is already unsustainable. This highlights the dire need for more efficient AI and ML technologies [**DeVries2023**]. The discovery of models that are sufficiently sustainable to curb this resource use is an open problem.

We believe that one path towards a solution is to combine different AI methodologies, in particular those technologies that are known for higher efficiency. One possible candidate would be the use of satisfiability solvers, in

particular modern Answer Set Programming languages, to accelerate the learning and inference processes.

2.2 Answer Set Programming (ASP)

Answer Set Programming serves as an amalgamation of Prolog-style (i.e., quantified logic) with powerful advances in satisfiability (SAT) solvers that occurred throught the 1990s and into the 2000s, expanding significantly in terms of the model sizes they are capable of processing and computational efficiency. In this section we will discuss the basic syntax and semantics of the ASP language, Clingo, which serves as a technological foundation of C-MPCN. **cite a relevant paper**.

2.2.1 Syntax and Semantics

The atomic units in an ASP dialect such as Clingo are *literals*, which describe ideas and *facts* that serve as statements that are held to be true. The basic syntax is the application of a *property* to a literal, `property(atom)`. E.g.,

```
red(apple).
```

is a complete statement (and program), attributing a property of red to an idea named `apple`. Such statements are *purely declarative*, providing a *description* of the problem (vs. specific processor instructions). I.e., we describe *what* needs solving vs. *how* to solve the problem in traditional imperative/algorithmic contexts.

Further knowledge is*inferred* using aggregate statements known as *rules*, taking the form: `consequent :- antecedent`. This mirrors an *implication* from formal logic, as the `:-` symbol is read equivalently to $\leftarrow$. In such a statement the consequent *must* be true whenever the antecedent is true. E.g., consider the statement:

```
eat(Fruit) :- red(Fruit), ripe(Fruit).
```

This describes a preference over what fruit will be eaten. I.e., "If a fruit is red and ripe, then we eat the fruit."

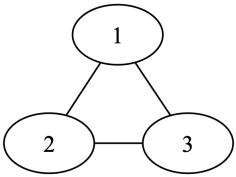
In this case rather than the literal `apple`, we use a *variable* to allow for inference. I.e., The `Fruit` can be *any* contextually-relevant literal in the program, so long as it possesses the required properties. Note that this functions differently from its analogous notion in imperative language–rather than a pattern to be matched variables typically serve as typed memory locations that are manipulated by the program.

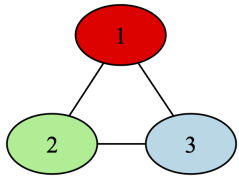
2.2.2 Problem Modeling in Clingo: 3-Colorability

As an illustrative example regarding how to model problems in Clingo, we will use the graph 3-Colorability problem, as it is presented in *ASP∀: Answer Set Programming "Algorithms"* (Guerin et al., 2026–need cite). ASP∀ is an Open Educational Resource (OER) for answer set programming in the Clingo dialect. These implementations provide minimal examples of how to model problems in Clingo, and graph-based implementations are of particular interest to us as ANNs are also graphical in nature.

The 3-Colorability problem is deceptively easy to state, but is computationally expensive to solve exactly. A simplified statement of the problems is "Given a graph, G, assign one of 3 colors to the vertices of G such that no two adjacent vertices share a color."

E.g., Consider the following graph and proposed solution:





Need to fix formatting.

In this particular case, color assignment is rather trivial–any assignment of all three colors to the vertices will lead to a valid coloring.

The implementation for 3-Colorability provided by ASP∀ is specified in terms of an *instance* as well as a *solution* file. The instance encodes input information in the form of predicates[1], and the solution provides a general implementation for inference over any provided instance model.

```
node(1..3).
node(1, 2).
node(1, 3).
node(2, 3).
```

[1]Unlike many programming language paradigms, ASP languages often lack traditional IO systems, moving these features to dedicated parsers in other languages.

```
% Set of available colors.
  color(1..3).

% Edges are symmetric.
% Assumes potentially directed description of an undirected graph.
edge(M, N) :- edge(N, M).

% Each vertex is assigned a single color.
1 { color(N, C) : color(C) } 1 :- node(N).

% No two adjacent vertices can have the same color.
:- edge(N,M), color(N, C), color(M, C).

% Output color information.
#show color/2.
```

```
clingo version 5.6.2

Reading from clingo_test.lp

Solving...

Answer: 1

color(1,2) color(2,3) color(3,1)

SATISFIABLE


Models : 1+
Calls : 1
Time : 0.013s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time : 0.002s
```

3 Methodology

3.1 Constructing the Logical Network

The bulk of our time on this project was spent on constructing the framework for logical networks, and testing it on an 8-bit adder. We chose this example because it is a simple problem that is still complex enough to demonstrate the capabilities of our framework.

A natural starting point for the framework was to create the structure of a sample graph, a 1-bit adder. It is easy to add too much information to each fact, and our final implementaion ended up with a much more minimalistic approach. Clingo is most effective with simple facts that encode a lot of information, marking one of the contributions our framework has to the field of both logic programming and neural networks.

For our example network we used neurons with three types of well established logic gates: AND, OR, and XOR. In Table 1 we show the definitions of these gates, and their truth tables. Using a combination of these and more gates, one is able to construct complex logical operations[2]. Moreover, we believe that the C-MCPN Framework has the potential to be used for a wide range of applications in the future, especially if its capabilities are further explored and developed.

AND			**OR**			**XOR**		
$g(\vec{x}) = x_1 \cdot x_2,\ \theta = 1$			$g(\vec{x}) = x_1 + x_2,\ \theta = 1$			$g(\vec{x}) = \lvert x_1 - x_2 \rvert,\ \theta = 1$		
Activation: $f(\vec{x}) = \begin{cases} 0 & \text{if } g(\vec{x}) < \theta, \\ 1 & \text{if } g(\vec{x}) \geq \theta. \end{cases}$								
AND			OR			XOR		
x_1	x_2	$f(\vec{x})$	x_1	x_2	$f(\vec{x})$	x_1	x_2	$f(\vec{x})$
T	T	T	T	T	T	T	T	F
T	F	F	T	F	T	T	F	T
F	T	F	F	T	T	F	T	T
F	F	F	F	F	F	F	F	F

Table 1: McCulloch-Pitts Neuron Definitions and Truth Tables.

3.2 Constructing the Editor

The framework is usable directly from the terminal, but we also built the C-MCPN Editor to make it more accessible and easier to use. The editor allows users to construct and visualize logical networks using the C-MCPN Framework. Installing and running the editor is relatively straightforward, and we provide detailed instructions in the README of the project.

When building this software we kept a few key design principles in mind. First, we wanted to make it as easy as possible for users to create and visualize

[2]An elegent and simple proof of Clingo's Turing completeness falls out of the C-MCPN Framework.

logical networks. Second, due to data ethics considerations we wanted the app to remain entirely local to the user's machine. Finally, we wanted to make it easy to extend the editor in the future to support a wider range of applications.

The editor is currently built in Python[3] using the DearPyGui library, allowing us to take advantage of its node-based editing capabilities. This meant we could keep the dependencies of the editor to a minimum, and focus on building the features we wanted.

Building the editor in this way also allowed us to easily integrate it with the C-MCPN Framework, as we could directly use the same two-layered data structures for both; one for the network architecture, and one for the values of each neuron. We also ensured that it was easy to add new gates to the editor. This was an important design choice, as it allows the editor to be easily extended in the future to support a wider range of applications. One more feature of note is the reorganization algorithm. We used a modified version of the Sugiyama algorithm, taking just the layer assignment step and a simplified node positioning step [**Sugiyama1981**].

3.2.1 AI-Assisted Editing

Claude (Anthropic) was used as a programming assistant throughout development of the editor, helping with DearPyGui module integration, debugging, and some code generation. All design decisions, research direction, and domain-specific logic (ASP, gate definitions, network structure) were made by the authors. Every line generated by Claude was reviewed, edited, and approved by the authors before being added to the codebase, and it did not have direct access to the codebase at any point. All of the files Claude had input in are contained in the `app/` directory of the project, though not all files in this directory had input from Claude.

4 Implementation

4.1 C-MCPN Framework

From what we could find in the literature, the C-MCPN Framework is a novel approach to neural network representation within a logical programming paradigm like ASP. The networks themselves have relatively simple representations that can be leveraged to create complex operations. The framework, which can be seen in Figure 1, uses three main program types:

- **Gate Definitions Program:** Files are in the `C-MCPN/gates/` directory.

[3]Python 3.8.5 is required (in part?) due to its native support for the tomlib package.

- **Network Structure Program:** Files are in the `C-MCPN/networks/` directory.
- **Value Inference Program:** Files are in the `C-MCPN/base/` directory.

The files in the gate definitions program define the behavior of each gate, including the number of inputs it should have. Every default gate we included also has a special header used to send meta-information to the C-MCPN Editor. For example, the file defining the AND gate looks like this:

```
%% gate: AND
%% inputs: 2
%% outputs: 1

% type("TYPE")
type("AND").

% gate(TYPE, OUTPUT, INPUTS)
gate("AND", Out, In1, In2) :-
    values(In1), values(In2),
    Out = (In1 * In2).
```

Note the metadata in the header, and the two rules which guide this gate's definition. The first simply defines the gate, and the second performs an operation on its inputs. Then, the files in the network structure program define the neurons which exist and their connections. For example, the file defining the 1-bit adder includes the following lines:

```
% neuron(type, unique_id)
neuron("INPUT", "INPUT_1").
neuron("XOR", "XOR_1").

% edge(source_id, target_id)
edge("INPUT_1", "XOR_1").

% { edge(source_id, target_id) : neuron(type, target_id) } max_inputs.
{ edge(src, "XOR_1") : neuron("XOR", "XOR_1") } 2.

% val(neuron_id, value).
val("INPUT_1", 1).
```

Note the cardinality constraints which enforce the correct number of inputs for each neuron, as well as the fact which sets the value of the input neuron

to 1. Finally, the value inference program performs two important roles. It activates each gate by allowing the edges between neurons to carry values, and it assigns each individual neuron a value based on its gate type. The following lines are from the value inference program, and show how this is implemented:

```
%% Value domain
values(0;1).

% Edges carry values from input to output
edge(In, Neuron_id, Val) :-
    edge(In, Neuron_id),
    neuron(_, Neuron_id),
    neuron(_, In),
    val(In, Val).

% 2 input gates
val(Neuron_id, Val) :-
    gate(Type, Val, Val1, Val2),
    neuron(Type, Neuron_id),
    edge(In1, Neuron_id, Val1),
    edge(In2, Neuron_id, Val2),
    In1 != In2.

%% Each neuron has one value
{ val(Neuron_id, Val) : values(Val) } 1 :- neuron(_, Neuron_id).

%% Show values
#show val/2.
```

Note the first fact which sets the domain of values to be binary and the constraint that ensures each neuron has exactly one value. Most importantly, the rule which carrys the values along the edges is vital to the functioning of the network. Without it, the network gets "stuck" and is unable to infer what the values of the neurons should be. This is a key insight we had during development.

4.2 C-MCPN Editor

In the `app/` directory of the project, we have the code for the C-MCPN Editor. The main features of the editor include the ability to add/remove neurons, connect/disconnect them, copy/paste them, and run inference on the network.

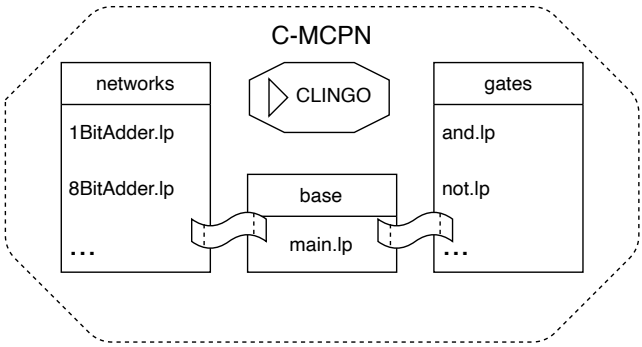


Figure 1: A visualization of the programs involved in the C-MCPN Framework.

The editor allows the files to be edited directly from the interface, transforming it into more of an interactive development environment (IDE) for logical networks. It includes a preview feature which allows users to see the structure of the network in ASP format before and after running the inference with simple syntax highlighting; helpful for debugging and understanding how the network is functioning. There is also an option to create new logic gates that the neurons can use, allowing for a wider range of applications.

Custom networks are saveable to JSON or ASP format and loadable from JSON format, making it easy to work on them over multiple sessions and share them with others. Saving the network to ASP format allows direct use with the C-MCPN Framework from the terminal instead of IDE.

The inputs for the inference use a Python script, allowing users to create complex inputs for their custom networks. The output can also be parsed with savable Python scripts built directly in the editor, allowing the user to choose how they want to process the results. Finally, it is worth noting that the editor includes a reorganization algorithm which automatically rearranges the neurons to make the network easier to read and understand after changes are made. This algorithm is based on the Sugiyama algorithm for layered graph drawing [**Sugiyama1981**].

A screenshot of the editor can be seen in Figure 2, showing a sample 1-bit adder network.

4.2.1 Files - General

There are a few folders and files to be aware of in this directory, including the following:

- `app/scripts/`: This folder contains the python scripts used to add the inputs and format the output from the ASP programs.
- `app/gui/`: This folder contains the code for the graphical user interface of the editor, explained in more detail in Section 4.2.2.
- `app/config.toml`: This file contains the settings for the editor.
- `app/main.py`: The main entry point for the editor, which initializes the GUI, establishes a few event handlers, and runs the application.
- `app/network_serializer.py`: Contains the network class used to store the structure and values of the logical networks in both JSON and ASP format.
- `app/tools.py`: Contains utility functions for finding the working directory of the project, and loading the config file and gate definitions.

4.2.2 Files - GUI

As mentioned earlier, this is the only folder with significant AI input. Most of that work was done in the `app/gui/` subdirectory where AI assistance was used mainly for integrating the DearPyGui library. This folder contains multiple modules which handle different aspects of the editor:

- `app/gui/gui_canvas.py`: Contains the code handling various canvas interactions, such as creating and connecting nodes.
- `app/gui/gui_files.py`: Contains the code for saving and loading networks from files.
- `app/gui/gui_output.py`: Contains the code for running the ASP programs and resizing the output to fit the editor's visualization.
- `app/gui/gui_preview.py`: Contains the code for previewing the network structure in ASP format before and after running the inference.
- `app/gui/gui_update.py`: Contains the code for updating the visualization and structure of the network when changes are made.
- `app/gui/gui_window.py`: Contains the code for resizing the editor window along with the dropdown for the gate editor.
- `app/gui/gui_tools.py`: Uses all of the previous modules to build the GUI and retrieve the gate list from the gate definitions. Here is where the various windows are created and organized.

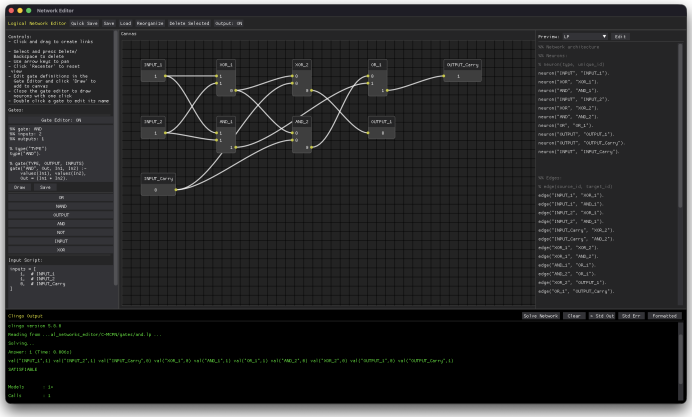


Figure 2: A screenshot of the C-MCPN Editor, showing a sample 1-bit adder network.

5 Evaluation

The 1-bit adder example network was used to evaluate the framework for correctness. Its truth table was compared to the expected output for all permutations of possible valid inputs. The verified network can be seen in in Figure 3.

The C-MCPN Framework works well for logical operations with a fixed number of inputs and outputs, and the C-MCPN Editor makes it easy to construct and visualize these networks. There were two key insights gained throughout the creation of this framework which are the importance of the edges carrying values and the minimalistic fact structure which improves the performance of the network. This framework is a novel approach to neural network representation within a logical programming paradigm like ASP.

The C-MCPN Editor also shows potential as a powerful educational tool for understanding both neural networks and logic programming. Showing a visualization of the network structure along with a live preview of the code necessary to create it is a great way to help users relate the two ideas.

5.1 Limitations

This work is a proof of concept for the C-MCPN Framework and Editor, and as such there are some limitations to be aware of. The framework has no cycle detection nor has it been tested on recurrent networks. This is likely to cause issues with the value inference program, as it relies on the values

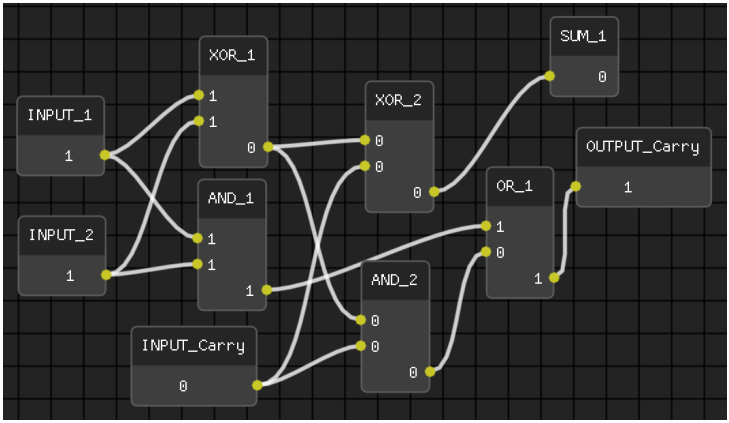


Figure 3: A screenshot of a 1-bit adder network in the C-MCPN Editor.

being propagated along the edges. The framework has also not been tested on large networks, and it is unknown how scale will affect its performance. It is also worth noting that Clingo does not have native support for floating point numbers, limiting the potential applications of easy to understand networks.

6 Conclusion

The C-MCPN Framework and Editor provide a novel approach to representing and working with logical networks within the paradigm of logic programming. The framework allows for the construction of logical networks using simple facts and rules, while the editor provides an interactive environment for building and visualizing these networks. Full instructions for using the editor can be found in the README of the project, and we encourage readers to try it out for themselves at `https://github.com/David-Gonzalez-Sua/logical_networks`. Importantly, the editor is designed with accessibility in mind, and we hope it can be used by a wide range of people to learn about these concepts.

6.1 Future Work

There is a lot of potential for future work in this area, both in terms of improving the C-MCPN Framework and Editor, and in exploring new applications for logical networks. For example, we have begun work on a new framework for an adaptive neural network which can learn on data that uses the C-MCPN Framework as its skeleton. This framework is in its early stages, but we hope to also add it as a feature in the editor in the future.